How I Cracked a Real LLD Interview

7 min read · Jun 21, 2025

Himanshu Singour Follow

Let me start with a quick real story.

I come from a **fintech background** I've worked on systems involving repayments, interest calculations, and customer accounts. So when I sat for an SDE-2 backend interview, I had a good idea the interviewer might go domain-specific.

And I was right.

After a short intro, the interviewer looked at me and said:

> "Himanshu, you're from a fintech company, right?
> Can you design a Loan Management System for me?"

I expected it.
And here's a pro tip for you **LLD interviews often depend on your domain**.

> *If you're from ed-tech, you might get "Design a Learning Portal."*
> *If you're from e-commerce, maybe "Design a Cart or Order Service."*
> *In my case, being from fintech,* ***Loan Management*** *was the natural choice.*

Though sometimes, interviewers also throw in common systems like BookMyShow, Hotel Booking, or Stack Overflow to test generic object modeling.

But this time it was real. It was my world.

And I knew this wasn't about writing code. It was about showing how I think like an engineer.

Here's exactly how I approached it from scratch to system.

Step 1: Clarify What They Actually Want

Before touching code or classes, I said:

Can I clarify the scope you want me to focus on?

Then I asked:

- "Should I include loan application or start after approval?"
- "Should I support fixed, floating, and stepped interest?"
- "Are we handling repayments, penalties, foreclosure, early closure?"
- "Who are the users customers or just admins?"

He said:

> Good questions. Let's keep it focused.
> Design the system *after* a loan is approved include scheduling, repayments, interest, charges, and closure.

Now I had clarity.
It wasn't a full loan lifecycle, it was just **loan servicing**.

Step 2: Identify the Entities, Relationships

I explained:

> Let me break this into real world objects I'll think like a product person first, not an engineer.

Entities:

- **Customer** → Person who owns the loan
- **LoanAccount** → Active loan record
- **Installment** → Every EMI (principal + interest)
- **Repayment** → Payment made by customer
- **InterestRate** → Fixed, floating, stepped
- **Charge** → Late fees, foreclosure charge, etc.

I told the interviewer:

> Every loan belongs to a customer.
> Each loan has a schedule with installments.
> Installments may or may not have repayments.
> Interest is configurable. Charges can be applied dynamically.

He nodded. I moved ahead.

Step 3: Class Design and Relationships

> *Here, you will face cross questions so be ready, especially in entity design. I faced almost 9 such questions; I've listed the ones I remember.*

Customer

```
class Customer {
    String customerId;
    String name;
    String email;
    List<LoanAccount> loanAccounts;
}
```

LoanAccount

```
class LoanAccount {
    String loanId;
    Customer customer;
    double principal;
    InterestRate interestRate;
    int tenureMonths;
    List<Installment> schedule;
    List<Charge> charges;
    LocalDate startDate;
    LoanStatus status; // ACTIVE, CLOSED, FORECLOSED
}
```

Installment

```
class Installment {
    int installmentNumber;
    LocalDate dueDate;
    double principalComponent;
    double interestComponent;
    double totalAmount;
    InstallmentStatus status; // DUE, PAID, LATE
    Repayment repayment;
}
```

Repayment

```
class Repayment {
    double amount;
    LocalDate paymentDate;
```

```
    String mode; // UPI, ONLINE, CASH
}
```

InterestRate

```
class InterestRate {
    InterestType type; // FIXED, FLOATING, STEP
    double baseRate;
    Map<Integer, Double> steppedRates; // optional
}
```

Charge

```
class Charge {
    ChargeType type; // LATE_FEE, FORECLOSURE
    double amount;
    LocalDate appliedDate;
}
```

CROSS QUESTIONS:

1. Why did you model `Installment` as a fixed list inside `LoanAccount`?
2. Can a single `Repayment` cover multiple `Installments`? How would your design support that?
3. How does your `InterestRate` model handle multiple mid-loan rate changes with history tracking?
4. Can your `Charge` entity differentiate between loan-level, installment-level, and recurring charges?
5. How would your design handle multiple active loans of different types under one customer?
6. In a multi-tenant system, is it safe to directly link `LoanAccount` to `Customer`?

Step 4: Service Layer (Where the Logic Lives)

I said:

“I’ll keep models dumb and move all logic to services. Services handle orchestration, calculation, and business logic.”

LoanService

```
class LoanService {
    LoanAccount createLoan(Customer customer, double principal, InterestRate ra
    void forecloseLoan(LoanAccount loan, LocalDate date);
}
```

ScheduleService

```
class ScheduleService {
    List<Installment> generateSchedule(LoanAccount loan);
    void updateScheduleAfterRateChange(LoanAccount loan);
}
```

RepaymentService

```
class RepaymentService {
    void applyRepayment(LoanAccount loan, Repayment repayment);
}
```

ChargeService

```
class ChargeService {
    void applyLateFees(LoanAccount loan);
    void applyForeclosureCharge(LoanAccount loan);
}
```

Step 5: Full API Design Based on Service Methods

Note:

In API design, always make sure to:

- Include the **HTTP method** (GET, POST, PUT, etc.)
- Clearly define the **endpoint URL**
- Provide the **full request body** (if applicable)
- Mention **query/path parameters** if any
- Add expected **response structure**
- Cover any **validations or edge cases**
- Explain briefly what each API does in **one line**

Remember:

Good:

- `POST /loan-accounts`
- `GET /loan-accounts/{loanId}/schedule`

Bad:

- `POST /CreateLoan`
- `GET /loanAccountSchedule`

1. LoanService

Method 1: `createLoan(...)`

API:

`POST /loans`

Request:

```
{
  "customerId": "CUST123",
  "principal": 500000,
  "tenureMonths": 24,
  "interestRate": 12.5,
  "interestType": "FIXED"
}
```

Response:

```
{
  "loanId": "LOAN789",
  "message": "Loan created successfully",
  "status": "ACTIVE"
}
```

Validations:

- Customer must exist
- Principal and tenure should be valid
- InterestRate type must be supported

Method 2: `forecloseLoan(...)`

API:

`POST /loans/{loanId}/foreclose`

Request:

```
{
  "foreclosureDate": "2025-11-01"
}
```

Response:

```
{
  "message": "Loan foreclosed successfully",
  "foreclosureAmount": 524000,
  "status": "FORECLOSED"
}
```

Validations:

- Loan must be ACTIVE
- All previous dues must be cleared
- Apply foreclosure charge before closing

2. ScheduleService

Method 1: `generateSchedule(...)`

API:

`GET /loans/{loanId}/schedule`

Response:

```
{
  "loanId": "LOAN789",
  "schedule": [
    {
```

```
        "installmentNumber": 1,
        "dueDate": "2025-07-01",
        "principal": 20000,
        "interest": 5200,
        "total": 25200,
        "status": "DUE"
      },
      ...
    ]
}
```

Method 2: `updateScheduleAfterRateChange(...)`

API:

```
PUT /loans/{loanId}/schedule/rate-change
```

Request:

```
{
  "newInterestRate": 13.0,
  "effectiveFromInstallment": 5
}
```

Response:

```
{
  "message": "Schedule updated successfully",
  "updatedFrom": 5
}
```

3. RepaymentService

Method 1: `applyRepayment(...)`

API:

```
POST /loans/{loanId}/repayments
```

Request:

```
{
  "amountPaid": 25200,
  "paymentDate": "2025-07-01",
  "mode": "UPI"
}
```

Response:

```
{
  "installmentNumber": 1,
  "status": "PAID",
  "message": "Repayment applied to installment 1"
}
```

Method 2 (Bonus): `getRepaymentHistory(...)`

API:

```
GET /loans/{loanId}/repayments
```

Response:

```
{
  "repayments": [
    {
      "installment": 1,
      "amount": 25200,
      "date": "2025-07-01",
      "mode": "UPI"
    },
    ...
```

```
    ]
}
```

4. ChargeService

Method 1: `applyLateFees(...)`

API:

```
POST /loans/{loanId}/charges/late-fee
```

Request:

```
{
  "installmentNumber": 2
}
```

Response:

```
{
  "message": "Late fee applied to installment 2",
  "amount": 500
}
```

Method 2: `applyForeclosureCharge(...)`

API:

```
POST /loans/{loanId}/charges/foreclosure
```

Request:

```
{
  "foreclosureDate": "2025-10-15"
}
```

Response:

```
{
  "message": "Foreclosure charge applied",
  "chargeAmount": 2000
}
```

Step 6: Concurrency & Edge Cases

I added without being asked:

- **Duplicate repayment** → use transaction ID to make it idempotent
- **Late payment** → `ChargeService` auto-applies penalty
- **Partial repayment** → store balance in `Installment`
- **Multiple users paying at once** → use row-level lock or optimistic locking
- **Interest rate changes mid-loan** → `ScheduleService` regenerates schedule from that date onward

> *Make sure to add concurrency, its very very very impl...*

Step 7: Ready for the Cross Questions?

Let's say you've just finished walking the interviewer through your Loan Management System design.

Now comes the real test **follow-up questions**. This is where they check:
"Is this person just repeating patterns... or have they really thought through the system?"

1. What if a customer tries to pay twice for the same installment? Will they be charged double?

Interviewer asked:

> "Let's say a customer pays via UPI. But they accidentally click twice. Now two payments hit your API for the same installment.
> What will your system do? Will it charge double?"

My Response (Himanshu):

> Great question this is a classic real-world scenario and happens a lot with UPI or slow networks.
> To handle this, I'll make my repayment API **idempotent** using a `transactionId`.
> Every repayment request must include a `transactionId`, and I'll store it in the database with each repayment record.
> Before applying the repayment, I'll check if that transaction ID has already been processed.

Then I added:

> "If it's a duplicate, I'll simply return the previous success response.
> This way, no double charge, no confusion, and full safety for the user.
> Also, the API becomes **reliable even under retries or network flakiness**."

The interviewer nodded. That's a .

2. Can this system support loans with step-up EMIs? Like low EMI in first year, then higher?

Interviewer asked:

> "In many banks, home loans are offered with 'step-up EMI' EMIs increase gradually as customer's income grows.
> Can your schedule generation logic handle that?"

My Response (Himanshu):

> "Yes, I've already considered that. That's why in my `InterestRate` class, I added a field called `steppedRates`, which is a `Map<Integer, Double>`.
> The key is the installment number, and the value is the interest rate applicable from that point onward."

Then I explained with an example:

> “Let’s say the loan starts with 12% interest for 12 months, and from month 13 it becomes 13.5%.
> I’ll store this as:

```
{
  "1": 12.0,
  "13": 13.5
}
```

In `ScheduleService`, during generation, I’ll check this map and apply the current rate based on installment number.”

Finally, I added:

> “This makes the system **future-proof** for real-world business use cases like balloon payments, holiday period, or variable rate loans.”

Another nod.

3. What if interest rate changes in the middle of the loan? Do you reprocess everything?

Interviewer asked:

> “Let’s say a bank decides to increase interest rate mid-loan due to policy change.
> What happens to future installments?
> Do you recalculate the full schedule again?”

My Response (Himanshu):

> “No, I won’t recalculate the already paid or overdue installments. I’ll only regenerate the future part of the schedule.
> I’ll maintain a new `InterestRate` object with an `effectiveFromInstallment` field.”

Then I explained my logic:

> “Let’s say the interest changes from EMI 10 onwards.
> In that case, I’ll:

- Keep the first 9 installments untouched

- Regenerate installment 10 onward using the new rate
- Update the schedule accordingly and mark affected EMIs as recalculated”

Bonus point I added:

> “To avoid breaking historical reporting, I’ll store both the original and revised schedule that helps in explaining differences in statements and audits.”

Step 7: SOLID Principles

> *Interviewer asked me have you applied SOLID principles in your design? If yes, where exactly?*

I Said, Yes

Single Responsibility Principle

Every class has one job.

➡ `LoanService`, `ScheduleService`, `RepaymentService`, all separated by responsibility.

Open/Closed Principle

Easy to extend, no need to change core logic.

➡Add new `InterestType` or `ChargeType` without touching existing code.

Liskov Substitution Principle

Future strategies can replace base logic without breaking flow.

➡Plug in new `InterestCalculationStrategy` safely.

Interface Segregation Principle

No bloated interfaces.

➡Can add interfaces like `Chargeable` or `Schedulable` only where needed.

Dependency Inversion Principle

High-level modules depend on interfaces.

➡Services depend on abstractions like `InterestCalculator`, not direct classes.

Step 8: Design Patterns

> *“Then the interviewer asked me what design patterns have you used, and where exactly did you apply them?*

Strategy Pattern

➡For different interest types (`FIXED` , `FLOATING` , `STEP`)
Used in `ScheduleService` to switch logic dynamically.

Factory Pattern

➡Returns the right strategy from `InterestStrategyFactory`
Keeps service code clean.

Builder Pattern (optional)

➡For fluent loan creation if object becomes complex
`LoanBuilder` makes it readable and test-friendly.

Full marks for business + tech awareness.

Low Level Design | Software Engineer | Corporate | System Design Interview | Lld

Follow

Written by Himanshu Singour

11.2K followers · 0 following

SDE-2 @ Fintech | Full-Stack | Top 1% in Global Contests (ICPC, Code Jam, Kick Start) | HLD/LLD & Passionate IN Distributed Systems

Responses (4)

Anubhav Gupta

What are your thoughts?

Surajbugade
Jun 22

Great ! Really helpful, got to know how the thought process should be.

1 Reply

NEY
Aug 19

Nice work. But where is the reference architecture before you start development? should that be first?

Reply

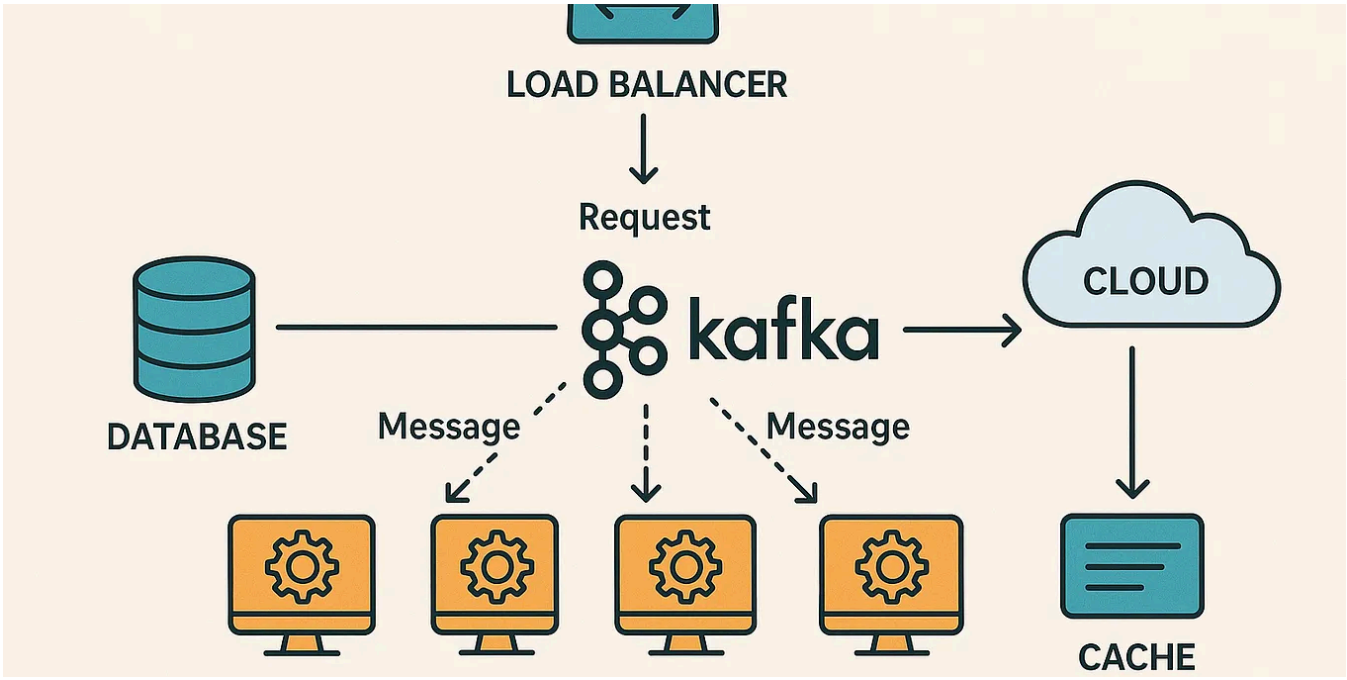
Kalpana
Jun 22

Thanks for sharing your experience..

Reply

See all responses

More from Himanshu Singour



Himanshu Singour

How I Learned System Design

– The honest journey from total confusion to clarity

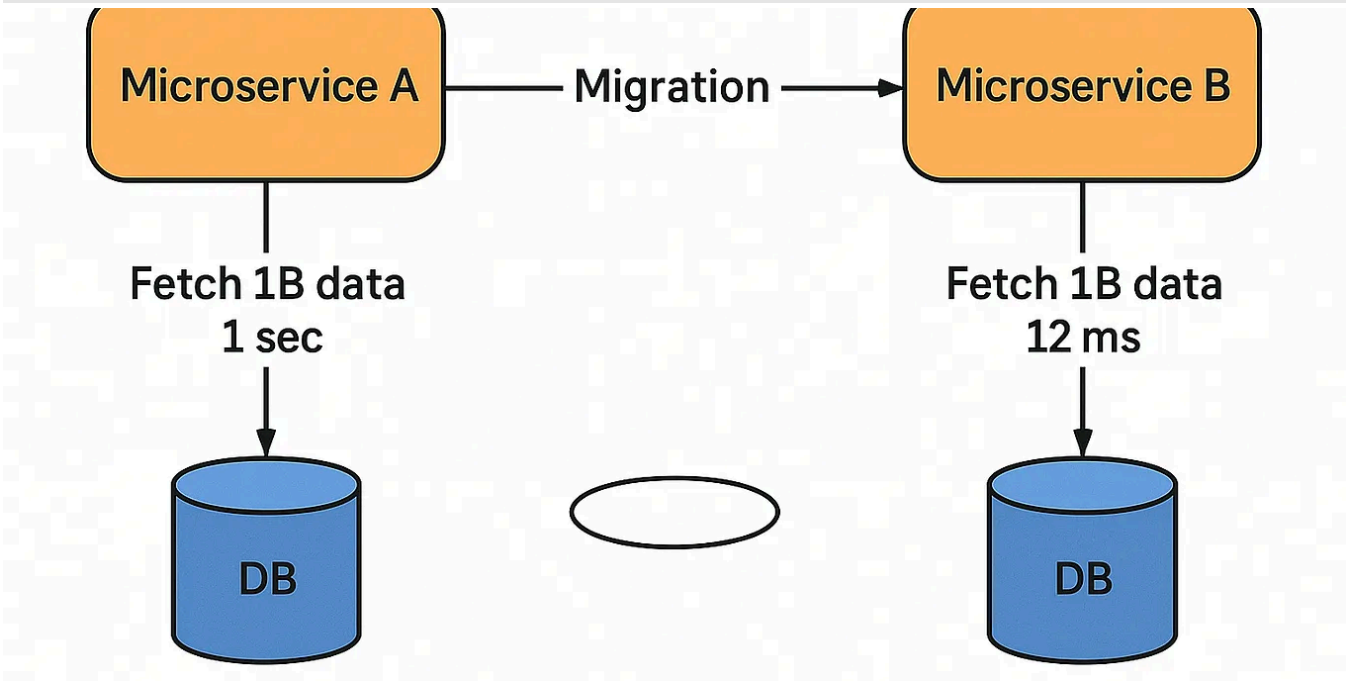
Aug 7 7.7K 219

Himanshu Singour

I Gave 2 Hours Daily to DSA & System Design, Best Decision Ever

I had 1.5 years of experience and still felt lost during interviews, confused in meetings, and unsure about my skills. I didn't need a...

Jul 10 8K 171

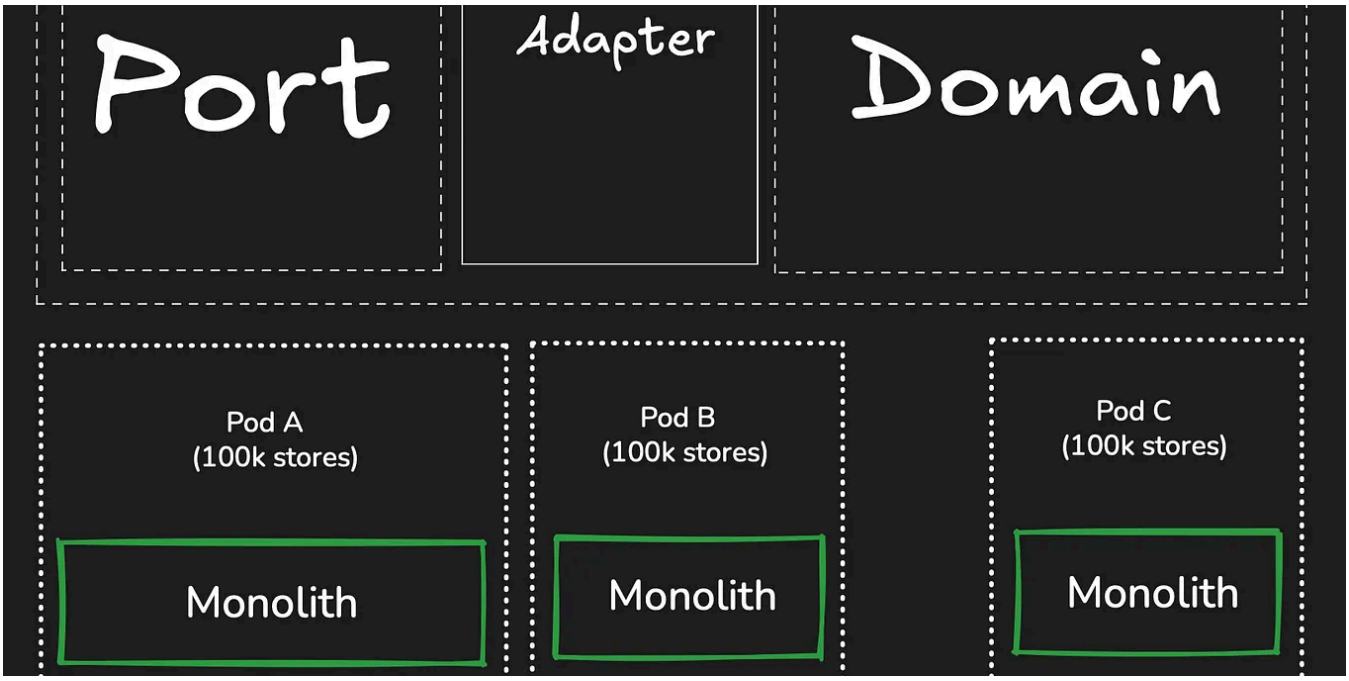


Himanshu Singour

How We Migrated DB 1 to DB 2 , 1 Billion Records Without Downtime

There are projects you never forget.

Sep 5 1.6K 46



Himanshu Singour

How Shopify Handles 30TB of Data Every Minute with a Monolithic Architecture

If you've ever worked on a web app that slows down with a few thousand users, try imagining one that handles billions.

Oct 9 1.2K 26

See all from Himanshu Singour

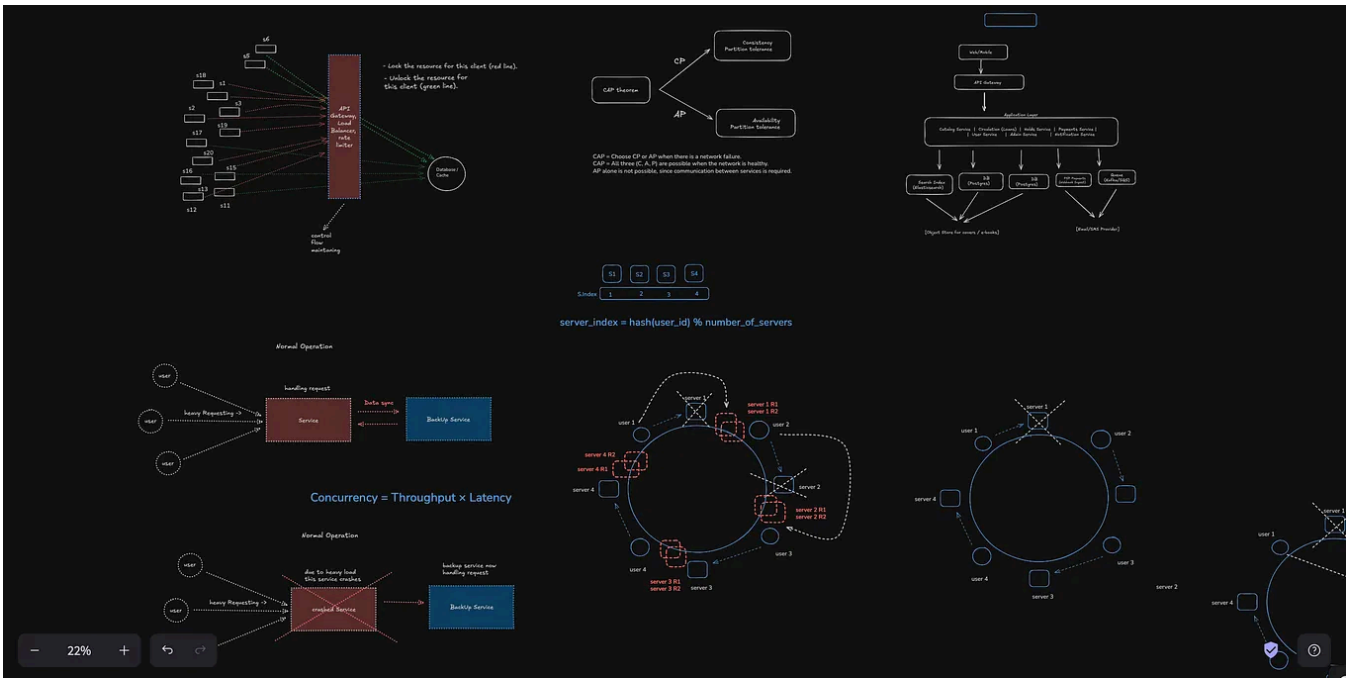
Recommended from Medium

Anant Singh Raghuvanshi

My SDE - 2 Interview Experience at Swiggy

Swiggy SDE-2 Backend Interview Experience: DSA Questions, System Design Rounds & Key Takeaways

Dec 15 141 10



Himanshu Singour

System Design Took Me from Missed Calls to ₹28 Base Package

The offer email came when I was in a crowded metro, half-asleep, hugging my backpack like it was a pillow.

Sep 29 427 8

Arvind Kumar

System Design Fundamentals & Framework—Your Foundation for Mastering Distributed Systems

Master the framework and core concepts that will help you tackle any system design question with confidence.

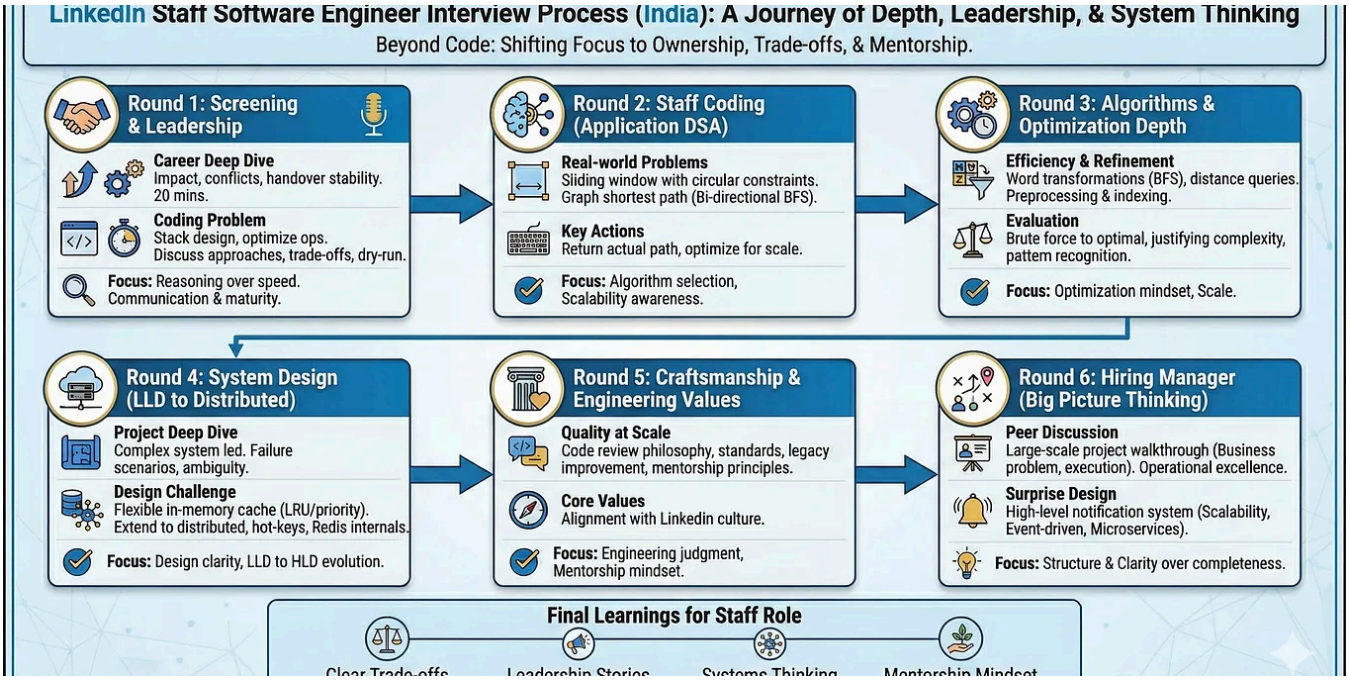
Dec 1 88 1

In Javarevisited by Firdaus Jawed

Most Important LLD Questions in 2025

Practice the top 10 Low-Level Design interview questions for 2025 with step-by-step solutions using “The System Design Framework”. Includes...

Sep 18 59 3

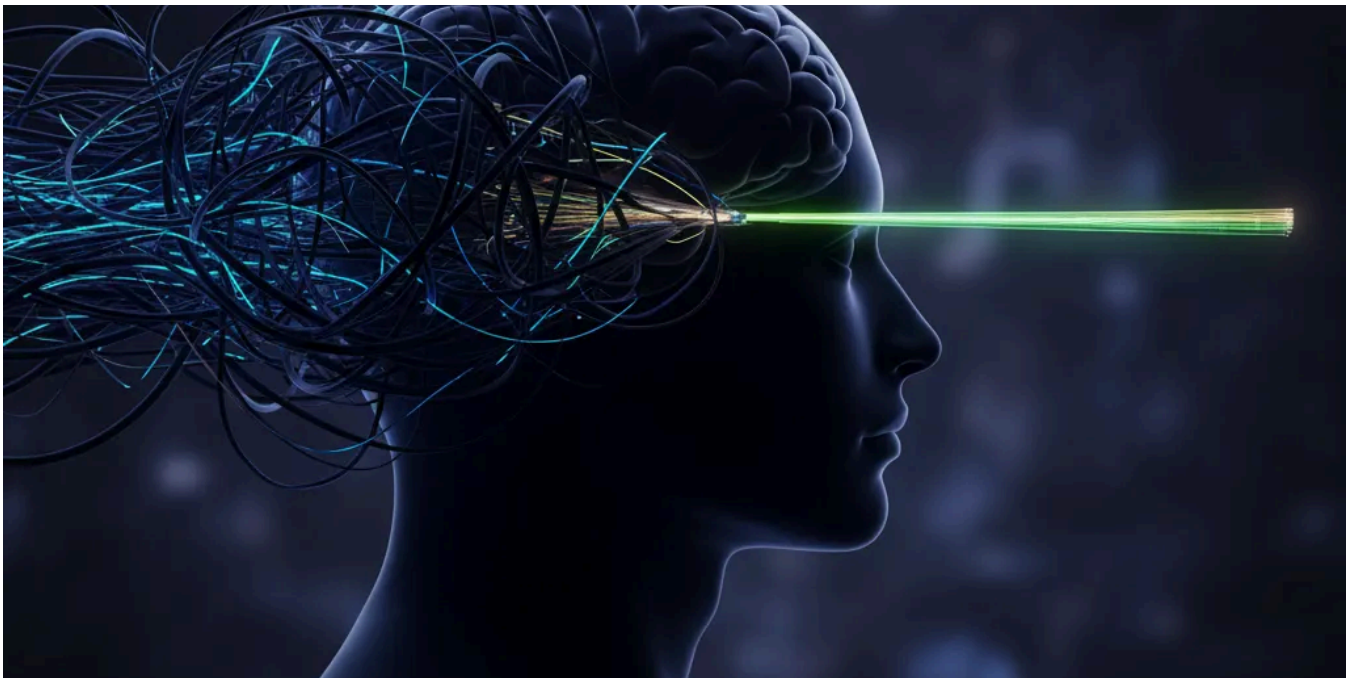


In Stackademic by Anurag Goel

LinkedIn Staff Software Engineer Interview Experience

A Realistic, In-Depth Journey Through the Process

Dec 14 70



In Beyond Localhost by The Speedcraft Lab

I Failed 47 System Design Interviews — Then One Netflix Engineer Changed Everything

A single shift in how I framed scalability turned rejection into three competing offers in thirty days.

Nov 5 2.2K 57

See more recommendations